1. **What is Apache Kafka?**
   Apache Kafka is a fast and reliable system that moves data between applications in real time. It collects data from different sources, stores it safely, and delivers it to other systems without losing it, even if failures happen. It's widely used for real-time processing like notifications, logs, and live data streaming.

2. **Main components of Kafka (short & simple):**
   - **Producer** → Sends (publishes) data to Kafka topics.
   - **Consumer** → Reads (subscribes to) data from topics.
   - **Broker** → Kafka server that stores and manages data.
   - **Topic** → Category/stream where messages are stored.
   - **Partition** → Splits a topic into parts for parallel processing & scalability.
   - **Zookeeper / KRaft** → Manages cluster metadata, leader election, configs.

   Communication in Kafka:

   Synchronous communication → Producer waits for Kafka acknowledgment after sending message (safer, slower).

   Asynchronous communication → Producer sends message without waiting for acknowledgment (faster, high throughput).

3) What is a Topic?

A Topic is a logical channel or stream in Kafka where messages (records/events) are stored. Producers send data to a topic, and consumers read data from it. Example: orders, payments, logs.

---

4) What is a Partition?

A Partition is a split of a topic. Each topic can have multiple partitions to allow parallel processing, scalability, and high throughput. Data inside a partition is always ordered.

---

5) What is Offset?

An Offset is a unique sequential ID assigned to each message within a partition. It helps Kafka track which messages a consumer has already read.

---

6) What does Kafka Producer do?

A Producer sends (publishes) messages/events to Kafka topics. It decides which topic and partition the message should go to (based on key or round-robin).

---

7) What is ACK in Producer?

ACK controls message durability and reliability:

- ACK = 0 → Producer does not wait for confirmation (fastest, risk of data loss)
- ACK = 1 → Leader broker confirms write (moderate safety)
- ACK = all (-1) → Leader + all replicas confirm (safest, no data loss)

---

8) What is Kafka Consumer Group?

A Consumer Group is a group of consumers working together to read a topic.
Each partition is read by only one consumer within the group, which enables load balancing and parallel consumption.

---

9) What happens if a Consumer fails?

Kafka automatically rebalances the consumer group.
Partitions assigned to the failed consumer are reassigned to other active consumers, so processing continues without stopping.

---

10) Auto Offset Commit vs Manual Commit

- Auto Commit → Kafka automatically saves offsets periodically (easy, but risk of data loss or duplication).
- Manual Commit → Developer controls when offset is saved (safer, ensures message processed before commit).

🔹 Replication & Fault Tolerance

Fault Tolerance: Fault tolerance means a system can continue working even if some components fail (like a server, broker, or network). It prevents data loss and downtime by keeping backup copies and automatically switching to them when failure happens.

11) What is Replication Factor?

Replication Factor: In Kafka, the replication factor defines how many copies of a partition are stored across different brokers. Its main purpose is **fault tolerance and data safety**. If one broker goes down, Kafka can still serve data from another replica, so no data is lost and the system keeps running.

Example: If Replication Factor = 3, then one partition will have **1 Leader and 2 Replica copies** stored on different brokers. The leader handles all reads and writes, while replicas stay in sync. If the leader fails, one of the replicas automatically becomes the new leader, ensuring high availability.

---

12) Leader and Follower?

- Leader → Handles all read/write operations.
- Followers → Replicate data from leader for backup.

---

13) What is ISR (In-Sync Replica)?

ISR = Replicas that are fully synchronized with the leader.
Only ISR replicas are eligible to become the new leader if the current leader fails.

---

14) What happens if Leader Broker crashes?

Kafka automatically performs leader election.
One of the ISR followers becomes the new leader, so system continues without data loss.

---

21) How Kafka guarantees Message Durability?

Kafka ensures durability using:

- Replication (multiple copies of data)
- ACK = all (waits for all replicas)
- ISR mechanism (only synced replicas become leader)

---

22) How to avoid Data Loss in Kafka?

Best practices:

- Replication Factor ≥ 3
- ACK = all
- Configure min.insync.replicas
- Use Manual Offset Commit
- Avoid unclean leader election

---

23) How to Scale Kafka Consumers?

To increase processing power:

- Increase number of partitions
- Add more consumers in the same consumer group

(Max parallelism = number of partitions)

---

24) Can multiple Consumers read the same Message?

Yes ✅ — if they belong to different consumer groups.
Each group gets its own copy of the message.

---

25) Kafka vs RabbitMQ

Feature	Kafka	RabbitMQ
Model	Distributed Log	Message Queue
Throughput	Very High	Moderate
Latency	Moderate	Very Low
Storage	Persistent (disk-based)	Queue-based
Communication	Pull model (consumer pulls)	Push model (broker pushes)
Use Case	Big data, streaming, analytics	Task queues, job processing

---

9) Kafka Cluster

A Kafka cluster is a group of multiple **brokers (servers)** working together to store and manage data. It provides **scalability, fault-tolerance, and high availability**. If one broker fails, others continue serving data.

10) Kafka Streams
Kafka Streams is a **Java library** used to process and analyze data in real time directly from Kafka topics. It helps with **filtering, transforming, aggregating, and joining streams** without needing extra systems.

11) Kafka Connect
Kafka Connect is a tool to **move data between Kafka and external systems** (like databases, Elasticsearch, S3, etc.) without writing much code.

- **Source Connector** → External system → Kafka
- **Sink Connector** → Kafka → External system

12) Zookeeper → Who replaced it & Why?

- **Zookeeper** was used to manage **cluster metadata, broker coordination, and leader election**.
- It is **replaced by KRaft (Kafka Raft mode)**.
- **Why replaced?**
    - Removes extra dependency (no separate Zookeeper needed)
    - Simpler architecture
    - Better scalability & performance
    - Easier maintenance

13) How Topic uses Round Robin for Partition?
When a producer sends messages **without a key**, Kafka distributes messages across partitions using **round robin**:

- Message 1 → Partition 0
- Message 2 → Partition 1
- Message 3 → Partition 2
- Message 4 → Partition 0 (cycle repeats)

This ensures **equal load distribution and parallel processing** across partitions.

---

14) Message Ordering

- Ordering guaranteed **only inside a single partition**
- Not guaranteed across multiple partitions

---

15) Idempotent Producer (Duplicate protection)

- Prevents duplicate messages during retries
- Enable: enable.idempotence=true

---

16) Exactly Once Semantics (EOS)

Kafka ensures **no data loss + no duplication**
Using:

- Idempotent Producer
- Transactions
- Proper offset commit

---

17) How Kafka is faster than traditional MQ?

- Sequential disk write (very fast)
- Zero-copy transfer
- Batching
- Compression
- Partition parallelism

(They love this question)

---

18) Retention Policy

Kafka **does not delete after consume**
Messages stored based on:

- Time (e.g. 7 days)
- Size (e.g. 10GB)

---

19) Consumer Lag

Difference between:

Latest offset – Consumer offset

Used to monitor slow consumers.

---

21. What happens if a Broker crashes?

- One **follower from ISR becomes new leader**
- No data loss if **replication + ACK=all**
- System continues working

---

22. Can duplicate messages occur?

Yes, during **producer retry or consumer crash**
Avoid using **Idempotent Producer + EOS**

---

23. Why Kafka is fast?

- Sequential disk write
- Batching + compression
- Zero-copy transfer
- Parallel partitions

1) What is Message Key in Kafka?

Answer: Message key decides **which partition** a message goes to. Kafka hashes the key to choose the partition. Messages with the **same key always go to the same partition**, which guarantees ordering for that key. If the key is null, Kafka distributes messages using round-robin (or sticky partitioning).

---

2) What is Log Compaction?

Answer: Log compaction keeps **only the latest value for each key** in a topic. Older duplicate records for the same key are removed in the background. It is used for **CDC, configuration topics, and state stores**, where only the latest state matters.

---

3) What is Unclean Leader Election?

Answer: If enabled, an **out-of-sync replica** can become leader when the current leader fails. This may cause **data loss** because the replica may not have the latest data. Therefore, unclean leader election is usually **disabled in production**.

---

4) What is min.insync.replicas?

Answer: It defines the **minimum number of in-sync replicas (ISR)** that must acknowledge a write when ACK=all is used. If enough replicas are not available, Kafka rejects the write to **protect durability** and prevent data loss.

---

5) commitSync vs commitAsync?

Answer: commitSync() is **blocking and safer** — it waits until the offset commit succeeds, reducing risk of duplicate processing. commitAsync() is **non-blocking and faster**, but if commit fails, it may cause duplicates since it does not retry automatically in order.

---

6) What is Consumer Rebalance?

Answer: Rebalance happens when consumers **join or leave a group**, causing Kafka to reassign partitions among active consumers. During rebalance, consumption pauses briefly (**stop-the-world**). Types: **Eager** (revoke all, reassign) and **Cooperative** (incremental, smoother).

---

7) What is Sticky Partitioning?

Answer: When no key is provided, modern Kafka producer **sticks to one partition for a batch** before switching. This improves batching and **increases throughput** compared to strict round-robin.

---

8) What is Rack Awareness?

Answer: Kafka spreads replicas across **different racks or availability zones** so that even if one rack/datacenter fails, other replicas still exist. This improves **fault tolerance and availability**.

---

9) What is Controller Broker?

Answer: One broker in the cluster acts as the **controller**. It manages **leader election, partition assignments, ISR updates, and cluster metadata**. If the controller fails, a new controller is automatically elected.

---

10) Kafka Storage Internals?

Answer: Kafka stores data in **log segments (append-only files)** with index files for fast lookup. It relies heavily on **OS page cache** for performance, making disk reads/writes efficient. Sequential writes make Kafka very fast and disk-friendly.

---

11) Producer Retries & Delivery Guarantee?

Answer: Producer can retry sending messages using retries and delivery.timeout.ms. Delivery guarantees:

- **At-most-once** → no retries, possible data loss
- **At-least-once** → retries enabled, possible duplicates
- **Exactly-once** → idempotent producer + transactions, no loss/no duplicates

---

12) What is Schema Registry?

Answer: Schema Registry manages **message schemas (Avro/Protobuf/JSON)** separately from data. It ensures **backward/forward compatibility** so producers and consumers can evolve without breaking each other. Commonly used with Confluent Kafka.

---

13) What is Dead Letter Queue (DLQ)?

Answer: DLQ is a **separate Kafka topic** where failed or unprocessable messages are sent. It helps in debugging and prevents blocking the main processing flow. Used in Kafka Streams and Kafka Connect.

---

14) Security in Kafka?

Answer: Kafka supports:

- **SSL/TLS** → encryption of data in transit
- **SASL** → authentication (who can connect)
- **ACL** → authorization (who can read/write which topic)
  Together they secure the Kafka cluster.

---

15) How to Monitor Kafka?

Answer: Important metrics:

- **Consumer Lag** (slow consumers)
- **Under-replicated partitions**
- **ISR shrink/expand**
- Broker health, throughput, errors
  Common tools: **Prometheus + Grafana, Burrow, Confluent Control Center**.

Kafka + Spring Boot

1. How do you integrate Kafka with Spring Boot?

Use **spring-kafka** dependency and configure producer/consumer in application.yml.

Dependency

xml Copy code

```xml
<dependency>
  <groupId>org.springframework.kafka</groupId>
  <artifactId>spring-kafka</artifactId>
</dependency>
```

application.yml

yaml Copy code

```yaml
spring:
  kafka:
    bootstrap-servers: localhost:9092

    consumer:
      group-id: my-group
      auto-offset-reset: earliest
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.apache.kafka.common.serialization.StringDeserializer

    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.apache.kafka.common.serialization.StringSerializer
```

2 How to create Kafka Producer in Spring Boot?

```java
@Service
public class KafkaProducerService {

    @Autowired
    private KafkaTemplate<String, String> kafkaTemplate;

    public void sendMessage(String msg) {
        kafkaTemplate.send("my-topic", msg);
    }
}
```

3. How to create Kafka Consumer in Spring Boot?

```java
@Component
public class KafkaConsumerService {

    @KafkaListener(topics = "my-topic", groupId = "my-group")
    public void consume(String message) {
        System.out.println("Received: " + message);
    }
}
```

4. What is @KafkaListener?

Annotation used to consume messages from Kafka topic asynchronously.

5. How to send JSON message using Kafka?

Producer Config

java Copy code

```java
@Bean
public ProducerFactory<String, Object> producerFactory() {
    Map<String, Object> config = new HashMap<>();
    config.put(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
    config.put(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG, JsonSerializer.class);
    return new DefaultKafkaProducerFactory<>(config);
}
```

Send Object

java Copy code

```java
kafkaTemplate.send("user-topic", new User(1, "Vedant"));
```

---

6. How to consume JSON in Spring Kafka?

java Copy code

```java
@KafkaListener(topics = "user-topic", groupId = "group")
public void consume(User user) {
    System.out.println(user.getName());
}
```

---

7. How to handle Kafka message retry?

Use Retry + Dead Letter Topic (DLT)

java Copy code

```java
@RetryableTopic(attempts = "3", dltTopicSuffix = "-dlt")
@KafkaListener(topics = "my-topic")
public void consume(String msg) {
    if(msg.contains("fail")) {
        throw new RuntimeException("Error");
    }
}
```

---

8. What is Dead Letter Topic (DLT)?

Failed messages after retries are sent to another topic → topic-name-dlt.

Used for debugging & reprocessing.

---

9. How to ensure message ordering in Kafka?

- Use single partition
- Or use same key for related messages

---

10. How to avoid duplicate messages in Kafka?

- Enable Idempotent Producer
- Use Manual Offset Commit
- Use Exactly Once Semantics

11. How to do Manual Offset Commit in Spring Kafka?

yaml Copy code

```yaml
spring:
  kafka:
    consumer:
      enable-auto-commit: false
```

java Copy code

```java
@KafkaListener(topics = "topic")
public void listen(ConsumerRecord<String, String> record, Acknowledgment ack) {
    System.out.println(record.value());
    ack.acknowledge();
}
```

12. What happens if Kafka Consumer crashes?

- Kafka rebalances
- Another consumer in same group takes partitions
- Processing continues from last committed offset

---

13. How to scale Kafka consumers in Spring Boot?

- Increase partitions
- Run multiple app instances with same group-id

---

14. Kafka Transaction in Spring Boot (Exactly Once)

```
kafkaTemplate.executeInTransaction(kt -> {
   kt.send("topic", "message");
   return true;
});
```

---

15. Common Kafka + Spring Boot Errors (Interview Favorite)

Error	Reason
Consumer not reading	group-id mismatch
Serialization error	Wrong serializer
Rebalancing again & again	Consumer too slow
Duplicate messages	Auto commit issue
Timeout / Broker not available	Kafka not running

Scenario-Based Questions (Very Important)

Q: Design Order Processing System using Kafka + Spring Boot
Flow:
Order Service → Kafka → Payment Service → Kafka → Notification Service

Q: How will you handle failed payment messages?
Retry → If still fail → Send to DLT → Manual fix.

Q: How to process 1M messages/sec?

- Increase partitions
- Use batch consumer